函数
概念
函数的定义：函数是一段可重复调用的代码块，用于实现特定功能。
语法：def 函数名(参数):
函数体
return 返回值
函数的作用：减少重复代码，提高代码复用性，模块化编程，提高可维护性。
函数的调用
调用语法：函数名(实参)
调用过程：解释器遇到函数调用，暂停当前代码执行，进入函数内部执行，执行完毕后返回结果，继续执行后续代码。
参数传递
参数传递的基本概念
参数是函数与外部交互的接口，通过参数可以向函数传入数据，实现灵活的功能。
参数传递的本质
参数传递是将实参的值赋给形参，形参是实参的副本，修改形参不影响实参。
参数传递的方式
位置参数
按照参数定义的顺序传递参数，调用时必须提供所有参数，顺序不能改变。
def add(a, b):
return a + b
result = add(10, 20)
print(result)
# 输出：30
关键字参数
通过参数名指定参数值，可以改变参数顺序，提高代码可读性。
def add(a, b):
return a + b
result = add(b=20, a=10)
print(result)
# 输出：30
默认参数
参数有默认值，调用时可以不提供该参数，使用默认值。
def greet(name, msg="Hello"):
print(msg, name)
greet("Tom")
# 输出：Hello Tom
greet("Tom", "Hi")
# 输出：Hi Tom
可变参数
*args
接收多个位置参数，打包成元组。
def sum(*numbers):
total = 0
for n in numbers:
total += n
return total
result = sum(1, 2, 3, 4, 5)
print(result)
# 输出：15
**kwargs
接收多个关键字参数，打包成字典。
def print_info(**kwargs):
for key, value in kwargs.items():
print(key, value)
print_info(name="Tom", age=18, city="Beijing")
# 输出：name Tom
# age 18
# city Beijing
参数传递的规则
参数顺序：位置参数 → 关键字参数 → 默认参数 → *args → **kwargs
参数传递的注意事项
可变对象作为默认参数：默认参数只在函数定义时创建一次，多次调用共享同一个对象，可能导致意外结果。
def add(lst=[]):
lst.append(1)
return lst
print(add())
# 输出：[1]
print(add())
# 输出：[1, 1]
解决方法：使用不可变对象作为默认参数，如 None，在函数内部创建可变对象。
def add(lst=None):
if lst is None:
lst = []
lst.append(1)
return lst
print(add())
# 输出：[1]
print(add())
# 输出：[1]
参数传递的性能
参数传递是引用传递，传递的是对象的引用，不是对象的副本，性能开销很小。
参数传递的最佳实践
参数设计：参数数量不宜过多，建议不超过 5 个；参数顺序：必选参数在前，可选参数在后；参数命名：使用有意义的参数名，提高可读性；参数类型：使用类型提示，提高代码可维护性。
def add(a: int, b: int):
return a + b
result = add(10, 20)
print(result)
# 输出：30
参数传递的常见问题
参数传递的本质是引用传递，传递的是对象的引用，不是对象的副本。
参数传递的性能开销很小，传递的是引用，不是对象的副本。
参数传递的最佳实践：参数设计：参数数量不宜过多，建议不超过 5 个；参数顺序：必选参数在前，可选参数在后；参数命名：使用有意义的参数名，提高可读性；参数类型：使用类型提示，提高代码可维护性。
返回值
返回值的概念
返回值是函数执行后，通过 return 语句返回给调用者的数据，用于传递函数的执行结果。
返回值的作用
传递结果：将函数的执行结果传递给调用者，供后续使用。
控制流程：通过返回值控制程序的流程，实现条件判断和循环。
提高复用性：返回值可以被多次使用，提高代码的复用性。
返回值的语法
def 函数名(参数):
函数体
return 返回值
返回值的类型
单个值：返回单个数据，如 return 10。
多个值：返回多个数据，用逗号分隔，自动打包成元组，如 return a, b。
无返回值：函数没有 return 语句，或 return 后没有值，返回 None。
返回值的使用
赋值：将返回值赋给变量，如 result = 函数名(参数)。
直接使用：直接使用返回值，如 print(函数名(参数))。
作为参数：将返回值作为其他函数的参数，如 print(函数名(参数))。
作为条件：将返回值作为条件判断，如 if 函数名(参数):。
作为循环：将返回值作为循环的迭代对象，如 for 函数名(参数):。
返回值的注意事项
return 语句：return 语句会立即结束函数执行，后续代码不会执行。
多个 return：函数可以有多个 return 语句，但只有一个会执行。
返回值的类型：返回值可以是任何数据类型，包括数字、字符串、列表、元组、字典、集合、对象等。
返回值的最佳实践
明确返回值：函数应该明确返回值，避免返回 None 除非必要。
统一返回值：多个 return 语句应该返回相同类型的值，保持一致性。
提前返回：使用提前返回减少嵌套，提高代码可读性。
def 函数名(参数):
if 条件:
return 值1
return 值2
返回值的常见问题
忘记 return：函数没有 return 语句，返回 None，可能导致意外结果。
多个 return：多个 return 语句返回不同类型的值，可能导致类型错误。
返回值的性能：返回值是引用传递，传递的是对象的引用，不是对象的副本，性能开销很小。
作用域
作用域的概念
作用域是变量可以被访问的范围，决定变量的可见性和生命周期。
作用域的分类
局部作用域：函数内部的变量，只在函数内部可见，函数结束后变量销毁。
全局作用域：模块级别的变量，在整个模块中可见，程序结束后变量销毁。
内置作用域：Python 内置的变量和函数，如 print、len、int 等，在所有模块中可见。
作用域的查找规则
LEGB 规则：Local（局部）→ Enclosing（闭包）→ Global（全局）→ Built-in（内置）。
查找顺序：从内层作用域开始查找，逐层向外查找，直到找到变量或到达内置作用域。
作用域的修改
global 关键字：在函数内部修改全局变量，如 global 变量名。
nonlocal 关键字：在嵌套函数内部修改闭包变量，如 nonlocal 变量名。
作用域的注意事项
变量遮蔽：局部变量可以遮蔽全局变量，导致意外结果。
闭包变量：闭包变量是引用传递，修改闭包变量会影响外层函数。
作用域的最佳实践
减少全局变量：尽量减少全局变量的使用，提高代码可维护性。
明确作用域：明确变量的作用域，避免意外修改。
使用 global 和 nonlocal：需要修改全局变量或闭包变量时，使用 global 和 nonlocal 关键字。
递归
递归的概念
递归是函数调用自己的过程，通过将问题分解为更小的子问题来解决问题。
递归的基本结构
递归的基本结构包括：基本情况（终止条件）和递归调用。
def 函数名(参数):
if 基本情况:
return 基本情况的结果
return 函数名(修改后的参数)
递归的特点
递归的特点：递归调用自己，通过将问题分解为更小的子问题来解决问题。
递归的优点：递归代码简洁，易于理解，适合处理递归结构的数据。
递归的缺点：递归可能导致栈溢出，递归调用开销较大，递归代码可能难以调试。
递归的应用
递归的应用：阶乘计算、斯波那契数列、汉诺塔问题、遍历树结构、遍历图结构、分治算法、动态规划、回溯算法。
递归的实现
递归的实现：递归函数调用自己，通过将问题分解为更小的子问题来解决问题。